



The AWS EKS Community Module

One module call creates the cluster, the IAM roles, the addons and the node group. It also creates an OIDC provider, a KMS key, a log group, two security groups, a launch template and an access entry, none of which you state.

IN THIS LECTURE

01 What a module is

One versioned call, many resources, in your state and your account.

02 What it creates

The OIDC provider, KMS key, log group, security groups, launch template and access entry.

03 What a default carries

Monthly charges, security posture and settings you never stated.

04 Reading a module

Three places that state what it creates before you apply.



MODULE CALL

A module call: your inputs, its resource declarations

You supply a name, a Kubernetes version, subnets and a node group description. The module declares the cluster, the IAM roles, the addons and the nodes, and they land in your state file and your AWS account.

main.tf

```
module "eks" {  
  source      = "terraform-aws-modules/eks/aws"  
  version    = "21.24.0"  
  name       = "demo"  
  kubernetes_version = "1.36"  
  vpc_id     = module.vpc.vpc_id  
  subnet_ids = module.vpc.private_subnets  
  eks_managed_node_groups = {  
    default = {  
      instance_types = ["t3.medium"]  
      desired_size   = 2  
    }  
  }  
}
```

Inputs

A name, a Kubernetes version, the subnets to place things in, and the node group you want.



The module

Resource declarations maintained in the open, fetched from a registry at the version you pin.



Resources in your account

The cluster, the IAM roles and policy attachments, the addons and the node group, declared in the module's code and recorded in your state file.



A version bump changes what the module creates

A release can add a resource, change a default or rename an input. Pinning the version keeps that change visible before it reaches the account.

THE DEFAULTS

Resources the module creates that you did not name

Called with a name, a version, subnets and a node group, the module also creates each of the following resources, with settings taken from its own defaults.

Resource	What it is for
 OIDC provider	Lets a pod assume a scoped AWS role of its own instead of borrowing the node’s permissions
 KMS key and alias	Encrypts Kubernetes secrets at rest, with the IAM policy that lets the cluster use the key
 CloudWatch log group	Receives control plane logs: API server, authenticator and audit on by default, kept 90 days
 Cluster and node security groups	Generated rule sets for control plane traffic and for node traffic
 Launch template	Defines how nodes boot, instead of accepting the template EKS would supply
 Access entry and policy association	Grants cluster administrator to whichever identity ran the apply



All six exist after the first apply

Their settings come from the module: log retention, key policy, security group rules, node boot configuration and the scope of the admin grant.

DEFAULT SETTINGS

A default can add a resource, or replace a decision

A module creates resources you never named, and it also sets values for settings you stopped stating. The cluster authentication mode is one of those settings.



The module default

the default

```
authentication_mode = "API_AND_CONFIG_MAP"
```

- ✓ Access entries work, which is the current mechanism for granting cluster access.
- ! The older ConfigMap path stays live alongside them, so two ways of granting access exist at once.



The narrower setting

stated explicitly

```
authentication_mode = "API"
```

- ✓ Access entries are the only way in, so every grant is an AWS API call you can audit.
- ✓ The ConfigMap path is off, so no grant can be made by editing a ConfigMap in the cluster.



Absent from your code

Modules and providers both create resources that appear in no configuration anyone reviewed.



Not absent from your account

They are billed to you, they carry your security posture, and you are the one paged when they misbehave.



A default is not visible in review

A review shows the inputs you wrote. The values applied for everything else appear only in the source and the plan.

ORDERING

Addon ordering is set by an input, not by the module call

Pod networking must be in place before nodes come up, or they register NotReady reporting that the CNI plugin is not initialized. Cluster DNS must come after, since it is a workload and needs nodes to run on.

